Porto. TechHub

P.PORTO

AI-POWERED SOFTWARE ENGINEERING

| DESENVOLVIMENTO FULL-STACK |

VITOR MANUEL SANTOS

ÍNDICE

- Programação assistida e desenvolvimento com agentes
- Preparação do contexto e planeamento da implementação
- Desenvolvimento Backend com IA
- Desenvolvimento Frontend e integração com APIs
- Investigação e resolução de bugs
- Refatoração e Design Patterns
- Compreensão de código legado
- Revisão e validação das alterações

FORMAS DE PROGRAMAR COM IA

Modo	Intervenção da IA	Papel do programador
Autocomplete	Sugere código durante a escrita	Seleciona e adapta as sugestões
Pair programming	Explica código e propõe alterações através da conversa	Fornece contexto e orienta cada alteração
Coding Agent	Explora o projeto, altera ficheiros e executa verificações, conforme as ferramentas e permissões disponíveis	Define o objetivo, os limites e valida o resultado

- A escolha depende da tarefa, do contexto disponível e do grau de autonomia pretendido.
- Exemplo na Matriz: completar uma validação, desenvolver um formulário ou implementar a criação de tarefas em várias camadas.

CICLO DE DESENVOLVIMENTO COM IA

Fase	Trabalho a realizar
Analisar	Ler a especificação e compreender o código existente
Planear	Identificar alterações, dependências e formas de verificação
Implementar	Aplicar alterações pequenas e coerentes com o projeto
Testar	Executar verificações e comparar o comportamento com os critérios de aceitação
Rever	Inspecionar o diff e decidir se as alterações podem ser aceites

IMPLEMENTAÇÃO DE UMA FUNCIONALIDADE COMPLETA

- Uma vertical slice implementa um comportamento através das camadas necessárias da aplicação.

Camada	Responsabilidade
Frontend	Apresentar o formulário e comunicar o resultado
API	Receber o pedido e validar os dados de entrada
Lógica de negócio	Verificar o projeto e as permissões do utilizador
Persistência	Guardar a tarefa e a associação ao projeto

IMPLEMENTAÇÃO DE UMA FUNCIONALIDADE COMPLETA

- **User Story**:
  - Como membro de um projeto, quero criar uma tarefa para organizar o trabalho a realizar.
- **Critérios de aceitação** exemplificativos:
  - O título é obrigatório.
  - O utilizador precisa de permissão para criar tarefas no projeto.
  - A tarefa fica guardada e aparece na listagem.
  - Se a operação falhar, a interface apresenta uma mensagem clara.

CONTEXTO NECESSÁRIO PARA IMPLEMENTAR

- O agente precisa de conhecer a funcionalidade e as decisões já tomadas no projeto.

Contexto	Informação relevante
User Story	Objetivo e critérios de aceitação
Arquitetura	Camadas, responsabilidades e dependências
Modelo de dados	Entidades, relações e restrições
Contrato OpenAPI	Pedidos, respostas e erros previstos
Código existente	Componentes reutilizáveis e convenções
Instruções do projeto	Regras de implementação e comandos de verificação

- Na Matriz: implementar a criação de tarefas exige compreender como funcionam os projetos, os membros e as permissões.
- O contexto deve ser relevante para a tarefa e consistente entre os diferentes documentos.

EXPLORAÇÃO DO PROJETO ANTES DA ALTERAÇÃO

- Antes de implementar, o agente deve localizar o código e identificar os padrões existentes.
- Prompt de exemplo disponibilizado no ficheiro **09 - Anexos**
- Resultado esperado: um diagnóstico apoiado no conteúdo real do repositório.

PLANEAMENTO DE UMA ALTERAÇÃO

- Um plano útil identifica alterações concretas e explica como verificar o resultado.

Elemento do plano	Exemplo na Matriz
Comportamento	Criar uma tarefa associada a um projeto
Código afetado	Receção do pedido, regras de negócio e persistência
Dependências	Identidade do utilizador e permissões no projeto
Casos de erro	Título inválido, projeto inexistente ou acesso recusado
Verificação	Confirmar criação, persistência e tratamento dos erros

- Prompt de exemplo disponibilizado nos anexos.

IMPLEMENTAÇÃO DO BACKEND POR RESPONSABILIDADES

- A implementação deve seguir a arquitetura escolhida na Aula 02.
- Exemplo numa arquitetura em camadas:

Componente	Responsabilidade
DTO de entrada	Representar e validar os dados recebidos
Controller	Receber o pedido HTTP e encaminhar a operação
Service	Aplicar as regras de negócio e de autorização
Repository	Consultar e guardar os dados
Entidade	Representar os dados e relações do domínio

- Prompt de exemplo disponibilizado nos anexos.

VALIDAÇÃO E AUTORIZAÇÃO NO BACKEND

- Cada regra protege um aspeto diferente da operação.

Verificação	Pergunta a responder
Autenticação	Quem está a fazer o pedido?
Validação dos dados	O título e os restantes campos são válidos?
Existência do projeto	O projeto indicado existe?
Autorização	Este utilizador pode criar tarefas neste projeto?
Persistência	A tarefa ficou associada ao projeto correto?

VALIDAÇÃO E AUTORIZAÇÃO NO BACKEND

- Na **Matriz**:
  - O Backend obtém a identidade através do mecanismo de autenticação.
  - O Backend verifica as permissões para o projeto indicado.
  - As respostas de sucesso e erro respeitam o contrato da API.
  - A interface apresenta mensagens adequadas ao resultado.
- Caso a verificar: um utilizador autorizado num projeto tenta criar uma tarefa noutro projeto sem permissão.
- A validação no Frontend melhora a experiência de utilização. O Backend tem de aplicar as regras a todos os pedidos.

VERIFICAÇÃO DO BACKEND

- Antes de integrar o Frontend, é necessário verificar o comportamento da API.

Cenário	Resultado esperado
Pedido válido e utilizador autorizado	Tarefa criada no projeto indicado
Título vazio ou inválido	Pedido rejeitado com o erro previsto
Projeto inexistente	Resposta coerente com o contrato
Utilizador sem permissão	Operação recusada, sem criar a tarefa
Consulta após a criação	Tarefa disponível com os dados guardados

- Prompt de exemplo disponibilizado nos anexos.

VALIDAÇÃO E AUTORIZAÇÃO NO BACKEND

- O Frontend deve seguir o protótipo e os padrões visuais existentes.
- Contexto a fornecer:
  - Wireframe ou imagem de referência.
  - User Story e critérios de aceitação.
  - Contrato da API.
  - Componentes e estilos já utilizados.
  - Regras de navegação e interação.
- Prompt de exemplo disponibilizado nos anexos.

INTEGRAÇÃO ENTRE FRONTEND E API

- A integração deve respeitar o contrato em ambos os sentidos.

Aspeto	Verificação
Endpoint e método HTTP	Correspondem à operação definida
Dados enviados	Campos, tipos e obrigatoriedade corretos
Projeto selecionado	Identificador associado ao pedido
Autenticação	Mecanismo existente aplicado ao pedido
Resposta de sucesso	Dados recebidos refletidos na interface
Resposta de erro	Tratamento adequado ao erro devolvido

INTEGRAÇÃO ENTRE FRONTEND E API

- Na **Matriz**:
  - O utilizador preenche o formulário.
  - O Frontend envia o pedido para a API.
  - O Backend valida e guarda a tarefa.
  - A interface usa a resposta para apresentar o resultado.
- Verificação essencial: a tarefa apresentada deve corresponder ao resultado confirmado pelo Backend.

ESTADOS DA INTERFACE

- Uma funcionalidade inclui o comportamento da interface durante toda a operação.

Estado	Comportamento esperado
Formulário inicial	Campos disponíveis e instruções claras
Dados inválidos	Mensagens junto dos campos relevantes
Submissão em curso	Indicador de progresso e prevenção de novos cliques
Sucesso	Confirmação e atualização da listagem
Erro	Mensagem compreensível e preservação dos dados
Lista vazia	Indicação de que ainda não existem tarefas

- Prompt de exemplo disponibilizado nos anexos.

VALIDAÇÃO DA FUNCIONALIDADE COMPLETA

- A funcionalidade está pronta quando o percurso do utilizador **funciona através de todas as camadas**.

- Percurso de verificação na **Matriz**:
  1. Entrar com um utilizador autorizado.
  2. Abrir um projeto e criar uma tarefa válida.
  3. Confirmar que a tarefa aparece na listagem.
  4. Recarregar a página e confirmar que permanece disponível.
  5. Experimentar dados inválidos e verificar as mensagens.
  6. Confirmar que a API recusa a operação sem a permissão necessária.

VALIDAÇÃO DA FUNCIONALIDADE COMPLETA

- **Revisão final**:
  - O comportamento cumpre os critérios de aceitação?
  - O Frontend e o Backend respeitam o contrato?
  - As alterações seguem a arquitetura e as convenções?
  - O diff contém apenas alterações relacionadas com a tarefa?
  - Existem verificações pendentes ou limitações conhecidas?
- Evidência esperada: resultados observados e verificações executadas que sustentem a aceitação da funcionalidade.

INVESTIGAÇÃO DE BUGS COM IA

- A investigação deve ligar o comportamento observado ao código responsável.

Etapa	Objetivo
Reproduzir	Encontrar os passos que provocam o problema
Recolher evidência	Observar pedidos, respostas, logs e estado da aplicação
Formular hipóteses	Identificar possíveis explicações
Investigar	Confirmar ou excluir cada hipótese
Corrigir	Alterar o código responsável pela causa
Verificar	Repetir o cenário e testar comportamentos relacionados

- **Papel da IA**: explorar o código, relacionar evidências e propor verificações.
- **Papel do programador**: confirmar o diagnóstico e avaliar a correção.

DESCRIÇÃO DE UM BUG

- Uma descrição útil permite reproduzir o problema e reconhecer a sua resolução.
- Prompt de exemplo disponibilizado no ficheiro **09 - Anexos**
- Acrescentar, quando disponível: versão do código, ambiente, mensagens de erro e evidência dos pedidos à API.

EVIDÊNCIA E HIPÓTESES

- Sintoma: a tarefa aparece apenas depois de recarregar a página.

Hipótese	Evidência a procurar
A listagem não recebe a nova tarefa	Estado da lista antes e depois da submissão
A cache mantém dados antigos	Mecanismo de atualização ou invalidação após a criação
Um filtro oculta a tarefa	Filtros ativos e propriedades da tarefa criada
O Frontend interpreta mal a resposta	Resposta real da API e código que a processa
A atualização da lista falha	Erros e fluxo executado após o pedido de criação

- As hipóteses orientam a investigação. A causa só fica confirmada quando a evidência explica o comportamento.
- Neste exemplo: a tarefa aparecer após recarregar sugere um problema de atualização da interface, mas ainda é necessário verificá-lo.

CORREÇÃO E TESTE DE REGRESSÃO

- A correção deve resolver a causa identificada e preservar os comportamentos relacionados.
- Para o exemplo da **Matriz**:
  - Atualizar a listagem através do mecanismo existente no projeto.
  - Apresentar a tarefa uma única vez.
  - Preservar os filtros e a ordenação definidos.
  - Manter o tratamento de erros da criação.
- **Teste de regressão**: reproduz o cenário que provocava o bug e verifica o comportamento esperado.

  Deve detetar o problema na versão anterior e passar após a correção.
- Prompt de exemplo disponibilizado no ficheiro **09 - Anexos**

REFATORAÇÃO COM IA

- A **refatoração** melhora a estrutura interna do código, preservando o comportamento observável.
- **Objetivos possíveis**:
  - Reduzir duplicação e complexidade.
  - Melhorar nomes e organização.
  - Separar responsabilidades.
  - Facilitar a compreensão e os testes.
- **Comportamentos a preservar**:
  - Resultados e respostas da API.
  - Regras de negócio e permissões.
  - Validações e tratamento de erros.
  - Alterações produzidas nos dados.
- Na Matriz: reorganizar o código de criação de tarefas deve manter as mesmas regras de acesso ao projeto.

OPORTUNIDADES DE REFATORAÇÃO

Sinal no código	Possível melhoria
Validação repetida em várias operações	Centralizar a regra partilhada
Método com várias responsabilidades	Extrair operações com objetivos claros
Condições difíceis de compreender	Simplificar expressões e nomear regras
Nomes ambíguos	Usar termos consistentes com o domínio
Dependências difíceis de testar	Isolar o acesso a serviços externos

OPORTUNIDADES DE REFATORAÇÃO

- Exemplo na **Matriz**: várias operações repetem a verificação das permissões de um membro sobre um projeto.
- Uma melhoria possível é reutilizar o mecanismo de autorização existente ou extrair uma responsabilidade comum.
- **Padrões de desenho**: pedir à IA que justifique o problema resolvido, a alternativa mais simples e a complexidade acrescentada.

COMPREENSÃO DE CÓDIGO LEGADO

- O código existente pode ter regras importantes que a documentação já não descreve.
- Antes de alterar, é necessário construir um mapa do seu funcionamento.

Aspeto	O que identificar
Pontos de entrada	Quem chama o código e em que situações
Dependências	Serviços, bibliotecas e módulos utilizados
Dados	Informação recebida, transformada e guardada
Efeitos	Escritas na base de dados, ficheiros ou chamadas externas
Regras	Condições, exceções e comportamentos especiais
Cobertura	Testes existentes e cenários sem verificação

- Prompt de exemplo disponibilizado nos anexos.

TESTES DE CARACTERIZAÇÃO

- Os testes de caracterização registam o comportamento atual de código existente, criando uma referência para alterações posteriores.
- Exemplo na **Matriz**: um módulo legado calcula as tarefas que cada utilizador pode consultar.
  - Podemos registar o resultado para:
  - Um membro com acesso a um projeto.
  - Um utilizador sem acesso ao projeto.
  - Um projeto sem tarefas.
  - Um utilizador associado a vários projetos.

TESTES DE CARACTERIZAÇÃO

- **Processo de trabalho**:
  1. Observar o comportamento com dados controlados.
  2. Compará-lo com os requisitos conhecidos.
  3. Criar testes para os comportamentos que devem ser preservados.
  4. Refatorar em pequenos passos.
  5. Reexecutar os testes e investigar diferenças.
- Um comportamento existente pode conter um bug. Os testes de caracterização ajudam a detetar alterações, mas os requisitos determinam se esse comportamento está correto.

EXECUÇÃO DE TAREFAS COM CODING AGENTS

- Um **Coding Agent** pode executar uma tarefa através de várias ações, usando as ferramentas e permissões disponíveis.
- Durante a execução, pode:
  - Explorar o repositório e consultar as instruções do projeto.
  - Identificar os ficheiros relevantes e propor um plano.
  - Alterar código em vários componentes.
  - Executar testes e analisar os resultados.
  - Corrigir problemas encontrados e apresentar o diff.

EXECUÇÃO DE TAREFAS COM CODING AGENTS

- O programador define:
  - Objetivo e critérios de aceitação.
  - Âmbito e restrições.
  - Autonomia concedida.
  - Condições para aceitar a alteração.
- A autonomia exige objetivos verificáveis e acompanhamento proporcional à tarefa.

ACOMPANHAMENTO DA EXECUÇÃO

- Os pontos de controlo permitem avaliar o trabalho antes de acumular alterações.

Momento	Informação a verificar
Após a análise	Código relevante, evidência e dúvidas
Após o plano	Âmbito, ficheiros afetados e verificações previstas
Durante a implementação	Progresso, desvios e bloqueios
Após os testes	Resultados, falhas e limitações
Na entrega	Diff e cumprimento dos critérios de aceitação

ACOMPANHAMENTO DA EXECUÇÃO

- Quando surgem dificuldades:
  - Uma hipótese contrariada pela evidência exige rever o diagnóstico.
  - Uma falha já existente deve ser distinguida de uma regressão.
  - Um bloqueio deve incluir o que foi tentado e o que falta resolver.
  - Uma alteração fora do âmbito exige reavaliar o plano.
- Instrução útil: “Se uma nova descoberta invalidar o plano, explica o impacto antes de continuar.”

REVISÃO DO DIFF

- O **diff mostra as diferenças entre versões** e permite inspecionar exatamente o que mudou.
- Questões de revisão:
  - As alterações resolvem a causa identificada?
  - Todos os ficheiros alterados estão relacionados com a tarefa?
  - As regras de negócio e as permissões foram preservadas?
  - Os testes verificam o comportamento esperado?
  - Alguma alteração removeu validações ou enfraqueceu testes?
  - Existem mudanças de configuração ou dependências por justificar?
- Prompt de exemplo disponibilizado nos anexos.

CRITÉRIOS DE CONCLUSÃO DE UMA ALTERAÇÃO

- Uma tarefa termina quando existe evidência suficiente para aceitar o resultado.

Dimensão	Critério
Funcionalidade	Cumpre os critérios de aceitação
Integração	Funciona com os componentes envolvidos
Código	Respeita a arquitetura e as convenções
Verificação	Apresenta resultados dos testes relevantes
Âmbito	Contém apenas alterações justificadas
Revisão	O programador compreende e aceita o diff

- Prompt de exemplo disponibilizado nos anexos.